Agent = Harness(Model):
Five Mechanistic Primitives, and What Is Not One

Chase Dovey
Average Joes Lab
https://cdovey.dev

Abstract

An agent is a model wrapped in a harness, and the harness has five parts. INPUT captures what comes in. CONTEXT assembles what the model is shown. MODEL INTERFACE sends the request and returns the response. OUTPUT carries the response out. CONTROL FLOW decides whether to go again, hand back or stop. We propose this as the architecture of every harness, and show each primitive as the smallest program that adds it. Everything else said about a harness is a design decision within one primitive or a production concern, persistence, safety and security, observability, resilience, performance and evaluation, asked of every primitive. The vocabulary is needed because harnesses change results as much as models do, so a result belongs to the pair and comparison needs the harness stated in parts; it lets research compare primitives by their implementations, and lets debugging isolate a failure to the primitives an implementation touches. We deconstruct the popular harnesses against the five: six from their sources at pinned versions, Claude Code, Codex CLI, opencode, pi, Hermes Agent and a LangGraph workflow, and a wider set of thirteen from their sources or documentation. Every component found has a home. What the deconstruction cannot show is which implementation of a primitive is better; the vocabulary makes that comparison well-posed, by holding the model and four primitives fixed and varying one, and we set out the comparisons that would both use the framework and test it. The minimal harnesses and the evidence for every assignment are published with the paper.

1 The Architecture

A language model is a function from a sequence of tokens to a distribution over the next token, applied until a stop token is produced:

$$\mathit{TokensOut} = \mathit{Model}(\mathit{TokensIn}).$$

Viewed from outside, the model keeps no state between calls, cannot read from or act on its environment, and cannot call itself again. Chat templates, tool-call formats and reasoning tokens are conventions for laying out *TokensIn* and reading *TokensOut*; models learn them in training and can learn to emit calls to tools [29], but the calls are carried out, and the conventions written and parsed, by something else.

That something else is the harness: everything in an agent that is not the model. It turns the world into tokens for the model and the model's tokens into effects on the world.

$$\mathit{Agent} = \mathit{Harness}(\mathit{Model}).$$

The boundary is at the model's weights and its sampling. Memory across sessions, tool use and knowledge of the date or the working directory are all on the harness side. The software has

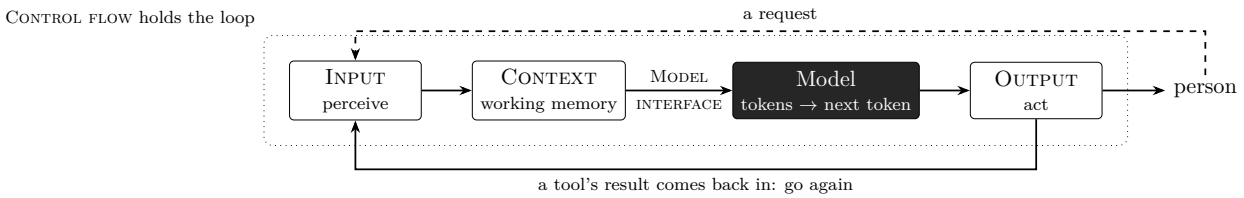


Figure 1: The agent loop. The five primitives surround the model; Control flow decides whether the loop runs again, hands back, or stops.

Algorithm 1 A harness. Lines are labelled with the primitive they implement.

```
1:  procedure ControlFlow( )
2:      conversation ← [ ]
3:      loop                                          ▷ outer loop: one request at a time
4:          conversation.append(User(Input( )))        ▷ Input: from a person
5:          loop                                      ▷ inner loop: one request, carried through
6:              ctx ← Context(conversation)           ▷ Context
7:              response ← ModelInterface(ctx)        ▷ Model interface
8:              conversation.append(Assistant(response))
9:              results ← Output(response)            ▷ Output: show words, run actions
10:             if results = ∅ then break             ▷ nothing ran: hand back
11:             conversation.append(User(results))    ▷ Input: from the world
```

many names: evaluations call it a *scaffold* [3, 20], libraries call themselves *frameworks*, *SDKs* or an *orchestration framework and runtime* [19], and *harness* is common among people who build coding agents [37]. None of the names says what the thing contains.

We propose that it contains five parts, and that the five are the same in every harness.

Input
: captures what comes in.

Context
: assembles what the model is shown.

Model interface
: sends the request and returns the response.

Output
: carries the response out: words to a person, actions on the world.

Control flow
: decides whether to go again, hand back, or stop.

Figure 1 shows how they sit around the model, and Algorithm 1 gives the loop they form, each line labelled with its primitive. Something comes in: a request from a person, an event, or the result of an earlier action. The harness assembles a context around it and sends the context to the model. The harness handles the response: it shows words to a person, or carries out the actions the response requests. If an action produced a result, the result comes back in and the loop runs again; if not, control returns to the person, or the agent stops. Every harness in §5 is an elaboration of this loop.

Everything else said about a harness is at one of two other levels. A *design decision* is one way of implementing a primitive: one tool or forty, a loop or a graph, a repository map or a summarizer. A *production concern* is a question asked of every primitive: what should outlast the session, what could do harm, what did it do, what if something breaks, what is slow, how do we know it still works. Neither adds a primitive (§3).

Scope. The primitives describe what happens on the path of a request at run time. Software that ships with a harness but never acts on that path is out of scope: hosting and transport, configuration loading, packaging, the user-interface framework, and the infrastructure that carries telemetry. The model is held fixed. A program that makes one model call and prints the reply has a harness, a trivial one; what makes a harness an agent's is a Control flow that feeds the results of actions back to the model.

What this paper does. §2 shows each primitive as the smallest program that adds it. §3 separates the primitives from design decisions and production concerns. §4 says why a shared vocabulary for the harness is needed and what it is for: comparing primitives by their implementations, and isolating a failure to the primitives an implementation touches. §5 deconstructs the popular harnesses against the five: six from their sources at pinned versions, and a wider set from their sources or documentation at summary level. §6 sets out the comparisons that would both use the framework and test it. The minimal harnesses are in Appendix A, the rules for the cases at the boundaries in Appendix B, and the evidence for every assignment in Appendix C.

2 The Primitives

Each primitive is defined by what it does and by what it never does. The negative definitions make the five mutually exclusive: every piece of a harness has one home, assigned by its function rather than by its name or its location in the code. Table 1 collects the definitions.

To show each primitive rather than describe it, we use five programs, each the previous one plus one primitive, all run on the same task: a two-file Python project whose `total()` subtracts a discount as a flat amount where the test expects a percentage, and the request *"The tests are failing. Find out why and fix it."* The model is Claude Sonnet through Anthropic's Messages API. The complete programs are in Appendix A; the excerpts below are the lines each primitive adds. In the transcripts, `>` is what the person typed, `<` what the model said, and `$` a command it ran.

2.1 Model interface: reach the model

Does. Send the assembled context to the model and bring back its response in a form the rest of the harness can read: words, and structured requests to act. The client, the wire format, the translation to the model's chat template, streaming, retries, and switching to a backup model when a request could not be served. **Never.** Decide what the request contains (Context). Act on the response (Output).

```
from anthropic import Anthropic

model = Anthropic()


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


response = request_response({"messages": [{"role": "user", "content": "The tests are failing. Find out why and
     fix it."}]})
print(response.model_dump_json(indent=2))
```

The model answers, but it can only answer: *"I don't have access to your code, test files, or test output. Nothing came through in your message besides the request."* The request is written into the program, so nobody can ask it anything else.

2.2 Input: capture what comes in

Does. Capture what enters the agent and hand it to Context. Two sources: a person (a message, a file, an interrupt, an answer to a question the model asked) and the world (the result of an action, an event, a scheduled trigger). **Never.** Decide what the model is shown (Context). Run anything (Output).

```
def capture_input():
    return input("\n> ")
```

Anyone can now type the request, and the answer is the same: *"I don't have the code or the test output yet, so I can't tell why the tests are failing. Could you share…"* It can only talk.

2.3 Output: carry the response out

Does. Handle what the model returned: show its words to a person, and carry out each requested action, collecting its result. The tool set, the executor that runs an action, and the environment it runs in. **Never.** Decide whether to go again (Control flow). Assemble the next request (Context). The model only ever *requests* an action; Output decides whether and how it happens, which is where sandboxes and never-lists live.

```
model = Anthropic()

bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["
     command"]}}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def capture_input():
    return input("\n> ")


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
```

A tool is described to the model, and `handle_output` runs each command the model asks for and collects the result:

```
> The tests are failing. Find out why and fix it.
< I'll start by exploring the project and running the tests.
$ pwd; ls -la; git status 2>/dev/null | head
```

Then it stops. The command ran and printed the files, and the result went nowhere.

2.4 Control flow: decide what happens next

Does. Define the shape of the loop: send results back and go again, hand back to the person, or stop. Step and token budgets, approval gates the harness consumes, delegation to sub-agents, and routing to a model keyed to the work. **Never.** Capture input, assemble context, call the model or run a tool. It sequences the other four.

```
def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
            response = request_response({"tools": [bash], "messages": conversation})
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
```

With the results sent back and the model called again until it stops requesting actions, the same model, tool and request finish the task:

```
> The tests are failing. Find out why and fix it.
< Let me start by looking at the project and running the tests.
$ pwd; ls -la; git status 2>/dev/null | head
$ cat prices.py test_prices.py; python -m pytest -q 2>&1 | tail -30
$ python -m unittest -v 2>&1 | tail -20
< The docstring says the discount is a percentage, but the code subtracts it
  as a flat amount. So 25 - 10 = 15 instead of 25 x 0.9 = 22.5. I'll fix the
  code to match the documented behavior.
$ sed -i 's|return round(subtotal - discount, 2)|return round(subtotal * (1 - discount / 100), 2)|' prices.py
   && cat prices.py && python -m unittest 2>&1 | tail -5
< The tests pass now. The bug was in prices.py, not in the tests.
```

This is an agent. Asked a question about itself, it has to go looking:

```
> Who are you, and where are you working?
< I'll check the environment first so I can answer accurately.
$ echo "user: $(whoami)"; echo "host: $(hostname)"; echo "cwd: $(pwd)"; uname -a; ls -la | head -20
< I'm an AI assistant. I can run shell commands, and I use that to read files,
  run code and make changes in the environment I'm given.
```

It does not know its name, its body, its working directory or the date.

2.5 Context: assemble what the model is shown

Does. Assemble, for one call, everything the model is shown: instructions, the tools' descriptions, the conversation so far, and anything retrieved from memory or the environment; and fit it to the window by truncation, compaction or selection. Context is the agent's working memory, rebuilt on every call. **Never.** Call the model (Model interface). Act on the world (Output). Reading a memory store is Context; writing one, if the model requests it, is an action and so Output.

```
import subprocess, os, datetime
from anthropic import Anthropic

model = Anthropic()
bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["
     command"]}}


def capture_input():
    return input("\n> ")


def assemble_context(conversation):
    instructions = f"You are quark, an agent. You act through bash, in {os.getcwd()}. Today is {datetime.date.
     today()}."
    return {"system": instructions, "tools": [bash], "messages": conversation}


def request_response(context):
```

Table 1: The five primitives. Each is defined by what it does and what it never does; the last column is what it hands to the next.

Primitive	Does	Never	Hands on
Input	captures what comes in, from a person or from the world	decides what the model is shown; runs anything	the captured item, to Context
Context	assembles what the model is shown on one call, and fits it to the window	calls the model; acts on the world	one request, to Model interface
Model interface	sends the request and returns the response as words and requests to act	decides what the request contains; acts on the response	the response, to Output
Output	shows words to a person; carries out each requested action and collects its result	decides whether to go again; assembles the next request	the results, to Control flow
Control flow	decides: go again, hand back, or stop	captures, assembles, calls, or runs	results back to Input, or control to the person

```
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
            print(f"$ {block.input['command']}")
            ran = subprocess.run(block.input["command"], shell=True, capture_output=True, text=True)
            tool_results.append({"type": "tool_result", "tool_use_id": block.id, "content": ran.stdout + ran.
     stderr or "(no output)"})
    return tool_results


def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
            response = request_response(assemble_context(conversation))
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
                break                                   # done: hand back to the person
            conversation.append({"role": "user", "content": tool_results})   # a tool ran: go again


control_flow()
```

One sentence of context, assembled on every call, answers the question without any action:

```
> Who are you, and where are you working?
< I'm quark, an agent that works by running shell commands through bash. My
  working directory is /tmp/shop. Today's date is 2026-10-07.
```

This is the whole harness: fifty lines, five functions, one per primitive. It runs commands on the host without asking and forgets everything when it exits; the production concerns of §3 are what it lacks.

Each primitive removes a failure the others do not. With only Model interface, the model answers but cannot act and nobody can ask it anything else; with Input, anyone can ask and it still cannot act; with Output, it acts once and the result goes nowhere; with Control flow, it finishes the task but spends actions finding out where it is; with Context, it knows. The same five programs, run with a local model in place of the hosted one, needed one function changed: `request_response`, which is Model interface. That is the porting property of §4 in practice.

3 Design Decisions and Production Concerns

Descriptions of harnesses mix three kinds of statement. "It has a loop" names a primitive. "It gives the model forty tools" names a decision within one. "It runs commands in a sandbox" names a measure taken for a reason that applies to every primitive. Keeping the three apart is the main use of the vocabulary. A primitive is a mechanism every harness has, defined by its function on the request path; a harness has exactly five. The other two levels are where harnesses differ and where they are made fit to run.

3.1 Design decisions

A design decision is one way of implementing a primitive. It is the level at which harnesses differ, and the level at which they are usually described. A design decision never adds a primitive; it fills one. The test for whether something is a design decision is whether a different harness could make a different choice and still be a harness. Table 2 lists the decisions found in the deconstruction of §5, by primitive, with the choices the harnesses made.

3.2 Production concerns

A production concern is a question asked of every primitive, whose answer changes how some primitives are implemented. We use six (Table 3): what should outlast the session (persistence); what could do harm, and who could make it (safety and security); what did it do (observability); what if something breaks (resilience); what is slow or costly (performance); and how do we know it still works (evaluation). A concern is not a part of the harness. A sandbox is not a sixth primitive; it is the safety answer in Output. A memory store is not a sixth primitive; it is a store in the world that Context reads and Output writes, kept for the persistence concern. A trace is not a sixth primitive; it is the observability answer, recorded wherever the primitive it watches runs. The test for whether something is a concern is whether it is motivated by a question that could equally be asked of a different primitive.

The concerns interact. Persistence creates a security surface: a poisoned memory steers every later session. Streaming changes safety: an interrupt can stop a reply mid-sentence, and the harness must record what was said rather than a finished answer. Summarizing changes persistence: the summary must say where the originals are kept. Treating concerns as questions asked of each primitive makes these interactions visible; treating them as separate components hides them.

3.3 Bundles and the cases at the boundaries

Much of what a harness's documentation names is a bundle. A mode, a skill, a hook system or an agent profile packages decisions across several primitives: a planning mode is a context template, a restricted tool set and an approval gate; a hook system is a dispatcher in Control flow whose events each affect one primitive. Decomposed, each element has one home. The negative definitions

decide most placements. Thirteen rules, each needed for a component found in §5, decide the rest, and are given in Appendix B. Three are used often enough to state here. A component is assigned by its function, not its location: an approval decision made inside a tool's runtime is CONTROL FLOW. A sub-agent is a harness of its own, and delegation to it is CONTROL FLOW of the outer harness. And one primitive may use another's mechanism to do its job: CONTEXT may ask the model to summarize old turns before compacting them, and the component is assigned by what it decides, with the call it makes a use of MODEL INTERFACE.

4 Why a Vocabulary

4.1 Harnesses change results as much as models do

If $Agent = Harness(Model)$, a benchmark score belongs to the pair. The evidence that the harness term is large is now direct. Holding GPT-4 Turbo fixed, Yang et al. [35] report 1.31% of SWE-bench [16] resolved with retrieval-augmented generation and 12.47% with their agent, and on SWE-bench Lite 11.00% for a shell-only agent against 18.00% for SWE-agent, with only the harness changing. Anthropic notes that performance "can vary significantly based on this scaffolding, even when using the same underlying AI model" [3]. METR found that tailoring a scaffold to one model more than doubled the length of tasks it could complete [20], and later that two mature harnesses did not outperform their default scaffolds [21], which suggests the effect is largest where the design decisions differ most. Zhang et al. [39] argue that harness configuration can account for more variance in performance than the choice of model, and ask that the harness be disclosed when agents are compared; Kapoor et al. [18] evaluate models, scaffolds and benchmarks as separate dimensions.

A disclosure needs a schema, and a comparison needs the harness stated in parts. Without one, two agents built on the same model are compared as wholes, and a difference between them cannot be attributed to anything smaller than "the scaffold". The five primitives are the schema: the input sources; what the context holds and how it is fitted to the window; the provider, decoding settings and retry policy; the tools, their output formats and the execution environment; and the loop's shape and stopping rules. Table 5 and Table 6 are such disclosures for nineteen harnesses.

4.2 Comparing primitives by their implementations

With the harness stated in five parts, a question that could not be asked becomes well-posed: which implementation of a primitive is better, for which model, on which task? The form of the study is to hold the model and four primitives fixed, vary the fifth, and measure. Fan et al. [10] do this for three decisions in a coding harness, planning, the action space and context management, across four models, and find that each changes behaviour. Their three dimensions were chosen independently of this paper and each falls within one primitive: planning in CONTROL FLOW, the action space in OUTPUT, context management in CONTEXT. The vocabulary says what the other dimensions are, and says which parts must be held fixed for a result to be attributed to the one varied. §6 lists the comparisons the framework makes well-posed.

The same property makes a harness portable. Because only MODEL INTERFACE knows which model is inside, a harness can be moved to a new model or provider by changing that primitive alone. A harness loses this property when model-specific details leak into other primitives: a provider's tool-call format assumed in OUTPUT, or a provider's error strings matched in CONTEXT management. The deconstruction records such leaks where it found them. It also says what a model trained for one harness must learn: the context layout it will be shown (CONTEXT), the format in

which it must request actions and the shape of the results it will read (Output), and the loop it runs in (Control flow).[1]

4.3 Isolating a failure

When an agent fails, the primitives give a short list of places to look, in order. The order matters: each question rules out a place the next question assumes.

1. Context**: was the model shown what it needed?** Read the request that was sent. If the file, the result or the instruction is not in it, the model could not have used it, and nothing downstream is at fault.
2. Output**: did what it asked for happen?** Compare the action the model requested with what ran. A parser that dropped a call, a sandbox that refused it, a tool that returned a truncated result, and an error message the model could not read are all Output.
3. Control flow**: did the loop stop early or run too long?** A step limit reached before the task was done, a hand-back on a response that still had work in it, and a loop that repeated the same action are Control flow.
4. Model interface**: was the response read correctly?** A tool call emitted in the right format but not recognized, a stream cut off and treated as complete, and a backup model substituted without notice are Model interface.
5. Input**: did the right thing come in?** A result delivered to the wrong turn, an interrupt lost, a person's answer consumed by the harness when the model needed it.
6. **Only then, the model.** If the model was shown what it needed, asked for the wrong thing, and its request was carried out as asked, the failure is the model's: its knowledge, its habits, or its size.

An implementation touches some primitives and not others, and a failure is isolated to the ones it touches. Two examples from our own transcripts. A small model, after instruction-tuning, sometimes emitted a second tool call as bare JSON without its `<tool_call>` tags. The harness read the JSON as words, found no action, and handed back. Question 4 locates this in Model interface, whose parser defines what counts as a call; the fix was on the model side, but the diagnosis was the harness's. In the same experiments, four kinds of task could never pass during reinforcement learning because the check compared the folder before and after, and the harness's own memory directory changed the folder on every run. No question in the list locates this, because the fault was in the evaluation, a concern, not in a primitive. Building the harness around a capable model first, until it passed every task in the evaluation, is what let every later failure be attributed to the model swapped in.

4.4 Existing vocabularies

The loop that harnesses implement goes back to ReAct [36], which interleaves a model's reasoning with actions whose results return to it as observations; in artificial intelligence more broadly, an agent perceives its environment and acts on it [28]. Later patterns live in particular primitives: Reflexion [31] writes reflections on failed attempts and carries them into the next, a decision in Control flow and Context; MemGPT [27] pages information in and out of a limited window, a decision in Context; Toolformer [29] trains the model to emit tool calls, moving policy into the

[1] In the companion book, a 0.6-billion-parameter model instruction-tuned on sessions recorded in its own harness rose from 2 to 18 of 32 held-out tasks in that harness, where the same base model post-trained by its developers for general use scored 14. The experiment, its data and its checks are described there [9]; this paper makes no claim that rests on it.

model while the calls are still carried out by OUTPUT; CodeAct [34] has the model write code as its action format, a decision in OUTPUT.

Sumers et al. [32] organize language agents by memory (working, episodic, semantic, procedural), an action space of external and internal actions, and a decision cycle. CoALA describes what an agent *is*, as a cognitive architecture; this paper describes what software an agent is *made of*. The correspondence is in Table 4. Two things here have no counterpart there: MODEL INTERFACE as a component, and the production concerns as a level of description distinct from the architecture. CoALA's memory modules, with the retrieval and learning actions that read and write them, are design decisions in CONTEXT and OUTPUT here, not a component.

Yang et al. [35] name the agent-computer interface, the commands a model is given, their output format and guardrails such as linting edits, and show it changes performance on software tasks; in our terms the interface is a set of design decisions in OUTPUT and CONTEXT, and this paper asks what the complete set of components is, of which the interface is part. Schluntz and Zhang [30] distinguish workflows, in which code fixes the sequence of model calls, from agents, in which the model directs its own process, and describe patterns built on an "augmented LLM"; we treat the distinction as a property of CONTROL FLOW, and decompose the augmented LLM into CONTEXT (retrieval, memory) and OUTPUT (tools) around a MODEL INTERFACE. Jin et al. [17] use "primitives" for recurring patterns of model calls in multi-agent systems (review, voting, planning and execution); in our terms these are shapes of CONTROL FLOW, and the two uses of the word are complementary. Tool protocols such as MCP [23] provide pluggable OUTPUT, with their other features assigned by function (Appendix B).

5 Deconstruction

If the architecture holds, a harness built by someone else, for a different purpose, at a different scale, should decompose into the five primitives with nothing left over, and its differences from other harnesses should be locatable as design decisions within a primitive. We test this on the popular harnesses, in two sets.

Selection. The first set is six harnesses in wide use whose primary sources were available, read in depth: two command-line coding agents from model developers (Claude Code [4] and Codex CLI [25]), two open-source coding agents built outside the model developers (opencode [2] and pi [38]), one general-purpose agent built for messaging platforms as well as the terminal (Hermes Agent [24]), and one workflow framework (LangGraph [19]), for which we deconstruct the canonical agent it ships, a ReAct loop, together with the runtime that executes it. The workflow framework tests the claim that a workflow and an agent differ only in CONTROL FLOW. The second set is thirteen more, read at summary level: the coding agents SWE-agent [33], OpenHands [1], Aider [11], Cline [7], Gemini CLI [13] and Goose [6]; the agent frameworks OpenAI Agents SDK [26], Google ADK [14], AutoGen [22], CrewAI [8] and smolagents [15]; and two closed harnesses read from their documentation alone, Cursor [5] and GitHub Copilot's coding agent [12]. Claude Code is likewise deconstructed from its documentation, since its source is not published. The nineteen are the harnesses we could read, not a sample of all harnesses; §5.8 says what that leaves untested.

Method. For each harness we (i) read the source at a pinned commit, or the documentation on a stated date, and mapped the path of a request from the moment a message arrives to the next model call; (ii) assigned every component on that path to exactly one primitive, or to one production concern together with the primitive it changes, by the definitions of §2 and the rules

of Appendix B, citing a file and line range or a page and section for each; and (iii) recorded every component that was hard to place and how it was resolved, or that it was not. For the first set every component on the path was assigned; for the second, two to five components per primitive, chosen as the ones that define the harness's design decision there. Components outside the request path (hosting, packaging, UI frameworks) were noted and left unassigned. The evidence is in Appendix C and in the repository as a file per harness; the subsections below summarize the first set.

5.1 Claude Code

Anthropic's coding harness, deconstructed from its documentation as read on 15 October 2026 [4]; its source is not published. The loop is the open one of Algorithm 1: the model is called until it returns a response with no tool calls, and a turn's result carries why it ended (done, a turn limit, a cost budget, an error). Input has one entry point for everything that starts a turn: typed prompts, piped input, messages queued mid-turn and delivered after the current tool batch, scheduled prompts, and reports from background sub-agents. Context is built as a stable cached prefix (the system prompt, the tools, then project files such as `CLAUDE.md`) with later updates appended as messages or system reminders; tool schemas for MCP servers are deferred behind a search tool; and compaction, on reaching the window, is a separate request that re-injects the project files, memory, recently read files and invoked skills. Model interface is the Messages API with streaming, up to ten retries that never re-issue after a completed tool block, idle watchdogs, and a fallback chain the user configures. Output is a fixed built-in tool set plus MCP tools; read-only calls run concurrently and mutating calls in sequence; shell commands run under an OS sandbox with a domain-allowlist proxy; large results spill to files; file edits are checkpointed for rewind. Control flow adds a six-step permission pipeline (hooks, deny rules, ask rules, the permission mode, allow rules, then a prompt), delegation through an agent tool to a depth of three, and, at scale, a model-written JavaScript workflow the runtime executes, which sequences model calls and is therefore Control flow by the rule on model-written programs.

Three components were hard to place. The web-fetch tool calls the model itself to condense a page before the main model sees it: the call is Output, the condensing is a Context decision, and the extra request is Model interface. The default permission gate in "auto" mode is a second model asked to classify each action: a Control flow gate implemented as a model call with its own stripped context. And a server-side variant of that classifier runs inside the API during the session's own requests, so the harness only consumes its verdict; we return to this in §5.7.

5.2 Codex CLI

OpenAI's terminal and IDE coding agent, at commit `f0c5077` [25], written in Rust. A session is one thread that receives typed operations over a channel; a dispatcher matches each one (a turn's input, an interrupt, an approval, the answer to a question, a compaction request) and runs each user turn as a task. Input is that channel: text typed during a running turn is queued as steering rather than starting a new turn, an inter-agent mailbox and asynchronous hook results merge into the same pending-input queue, and the model's questions to the person come back as a typed answer. Context is the Responses API's own item list, with the environment and instructions injected as typed "world-state" sections sent as diffs when they change, project files walked from the root to the working directory, and tool outputs truncated in the live history only; compaction has three implementations (a local summary, a remote one performed by the server, and a token-budget reset), chosen by explicit threshold arithmetic, and can run speculatively in the background once usage passes a percentage of the window. Model interface is the Responses

API only, WebSocket first with a session-wide fallback to HTTP, five retries with jittered backoff that honour the server's retry timing, an idle timeout, and requests sent without server-side storage, with encrypted reasoning replayed. OUTPUT routes every risky action through one orchestrator (approval, then the platform sandbox, then the run, then a retry if the sandbox denied it), with Seatbelt on macOS, seccomp with bubblewrap and Landlock on Linux and a restricted token on Windows; tools run concurrently under a read–write gate that lets parallel-safe tools overlap; and a "code mode" runs model-written JavaScript in V8, each nested tool call re-entering the harness's approval and sandbox path. CONTROL FLOW is one task per turn, looping until the model emits no tool call, with stop hooks, roll-over into compaction mid-turn, an approval precedence of hooks, then an optional model reviewer ("Guardian"), then the person, and sub-agents that are full sessions of their own with depth and thread caps.

Hard cases: the Guardian is a CONTROL FLOW gate whose verdict comes from a nested harness; the remote compaction is a CONTEXT decision whose summarizer is the provider; over WebSocket the request omits the tools and instructions the server already holds, so what is sent depends on transport state while what the model is shown does not; and the plan-mode parser, which reads the streamed text into a proposed-plan item and can redirect it, is OUTPUT rendering that also feeds the stop decision. We record the last as straddling OUTPUT and CONTROL FLOW by design.

5.3 opencode

An open-source coding harness by Anomaly, at commit `654b79b` [2]; it was rewritten since earlier versions, and the deconstruction describes the current one. Each session is an event-sourced record in SQLite, and one fibre per session drains a durable inbox: INPUT is a row in that inbox, from a person, a shell command, a skill, or the harness itself, marked for delivery either at the next step boundary ("steer") or after the current turn ("queue"). CONTEXT re-projects the event history into model messages on every call, with instructions placed in a cached prefix and changes to them frozen into the history as dated events; compaction is triggered at about ninety percent of the window, using provider-measured token counts plus a local estimate, keeps the newest fifteen thousand tokens verbatim, and can be delegated to the provider's native compaction. MODEL INTERFACE is an in-house request type compiled per provider protocol (Anthropic Messages, OpenAI Responses, Bedrock Converse, Gemini), streamed over HTTP or a stateful WebSocket, with a retry policy split by whether output had already started. OUTPUT runs tool calls concurrently as they stream, and most tools are reachable only through one `execute` tool that runs model-written JavaScript in an interpreter that cannot call the model, which the rule on model-written programs places in OUTPUT. CONTROL FLOW continues while a step made tool calls, stops on a text-only finish, enforces an optional per-agent step budget, and delegates to child sessions (depth one by default), whose results return as tool text or as a synthetic message.

Hard cases: every tool asserts its own permission, so an approval gate the harness consumes lives inside the tool's code (CONTROL FLOW by function); the step budget is enforced by appending an assistant message and disabling tools, a CONTROL FLOW decision carried out by editing what the model sees; and the compaction service both decides what to summarize and issues the model call itself. Native provider-side compaction leaves the harness's CONTEXT holding an item it cannot inspect.

5.4 pi

A terminal coding agent by Mario Zechner, at commit `5ab7fe7` [38], built in layers: a minimal agent loop, a provider layer, and a coding-agent session on top. INPUT comes from a terminal

editor, a print mode, or a JSON-RPC mode, all into one `prompt()` call, with mid-run messages split into steering (after this turn) and follow-ups (after the agent would have stopped). CONTEXT is a projection of a persisted session tree: a compaction entry, the kept tail, and context edits, with the system prompt kept as named sections and diffed into patches, so that replaying the file reproduces the exact request; compaction triggers when the context exceeds the window less a reserve, and keeps the most recent twenty thousand tokens. MODEL INTERFACE is one streaming function behind about fifteen provider adapters, with the SDKs' own retries disabled so that the harness owns a two-tier retry policy, and an overflow classifier that routes a too-long request to compaction rather than to a retry. OUTPUT is a set of plain tool objects run in parallel by default, with mutations to one file serialized; there is no sandbox in the core (OS sandboxing and permission prompts are extensions), and a "code mode" tool runs model-written scripts in a QuickJS sandbox. CONTROL FLOW is a two-level loop (tool calls and steering inside, follow-ups outside) plus a session-level loop for retries, overflow recovery and extension continuations; sub-agents are not in the core, though an example spawns child processes; and per-request routing between "virtual models" is keyed to the reason for the request.

Hard cases: a wire-level hook lets an extension rewrite the request body, which is CONTEXT by function though it sits in the adapter; the virtual-model router has a "retry" branch that overlaps the backup-model rule (we place it in CONTROL FLOW, since the router decides by the state of the work); and the code-mode script can call auxiliary models from inside the sandbox, a model call issued from inside a tool, which we discuss in §5.7. A telemetry interface exists, but no span is emitted at this commit.

5.5 Hermes Agent

A personal agent by Nous Research, at commit `7bf3766` [24], with one agent core behind a terminal interface, about twenty-five messaging-platform adapters, an HTTP server, an ACP adapter and a cron scheduler. INPUT is one normalized message event shared by all of these; mid-turn messages become steering rather than new turns. CONTEXT is a three-tier system prompt, kept byte-stable so that caching holds: identity, then project files (capped by size), then volatile memory, skills and the time; compaction at half the window prunes, then summarizes with an auxiliary model; oversized tool output is spilled to files rather than dropped; and a tool-search bridge lets the model fetch tool schemas on demand. MODEL INTERFACE is an OpenAI-style chat-completions client by default with pluggable transports for Anthropic, Responses and Bedrock, streaming on, three retries with backoff, then a configurable fallback chain, then a bounded auto-recovery wait. OUTPUT sends a fixed core tool set on every call, splits each batch into parallel-safe segments, runs terminal commands behind pluggable sandboxes (local, Docker, Modal, and others) and a regex-plus-human approval gate, and persists each tool-call row before executing it. CONTROL FLOW is a loop of named phases, each returning a verdict, with an unlimited iteration budget by default, "stop-gate" nudges that append a synthetic message and go again, flat delegation to threaded children, and task-keyed routing to auxiliary models.

Two components do not fit cleanly. In one mode the whole turn is handed to an external Codex process, and Hermes keeps only input, persistence and a fallback: the five primitives collapse into one opaque external harness that is neither a sub-agent (there is no outer loop) nor the model. And "providers" reached over ACP make the thing behind the MODEL INTERFACE another agent. Both are nested harnesses by the rule of Appendix B, but of an unusual shape: the nesting is at the MODEL INTERFACE, not in CONTROL FLOW.

5.6 A LangGraph workflow

LangGraph [19], at commit `5eb60ae`, is a framework for writing control flow as a graph over a checkpointed state, executed by a runtime in bulk-synchronous supersteps. We deconstruct the agent it ships, `create_react_agent`, together with that runtime. Input is a write to a named channel of a typed state, merged by a user-chosen reducer, whether it is a person's message, a tool result, or a resume value after an interrupt. Context is exactly the state's message list, optionally prefixed by one system message; fitting it to the window is left entirely to a user-written hook before the model call, and the framework provides only a primitive for removing messages. Model interface is one call to a LangChain chat model per step, and the framework contains no client of its own, no streaming of its own, and no backup model; retries and timeouts are per-node policies, off by default. Output is a tool node that runs every tool call of a response concurrently in-process, turns results and (configurably) errors into messages, and lets a tool return a routing command. Control flow is the graph: "go again" is a conditional edge from the agent node to the tool node and back, "stop" is an empty task set or a recursion limit, and a pause is an exception the runtime persists as a checkpoint write and resumes by re-running the node.

This is the clearest case of the three-level distinction. The framework fixes the five primitives' mechanisms and leaves most of the design decisions to user code: the edges, the stopping rule, the window-fitting hook, the approval gate around an interrupt, dynamic model selection, retry policies. A workflow and an agent built with it differ in the graph, which is Control flow, and nowhere else. The hard cases are the step counter injected into the state (it looks like Context but no model sees it; it is a Control flow budget), and a scheduling packet that carries a state snapshot to the tool task (Control flow carrying Context).

5.7 Summary

Table 5 gives the six side by side, one row per primitive, and Table 6 the thirteen of the second set. Every component we found has a home. The differences between the harnesses are in the rows: what comes in and from where, what the context holds and how it is fitted, how the model is reached, which tools exist and where they run, and what shape the loop has. None of the nineteen required a sixth row.

Table 6: Thirteen more harnesses by primitive, read from their sources or documentation at summary level. Each cell is the design decision that harness made within the primitive; the evidence for each is in Appendix C, which also gives the pinned commit or documentation date.

	Input	Context	Model interface	Output	Control flow
SWE-agent	single problem statement (text, file, GitHub issue, images) plus templated bash observations; no mid-run messages	Jinja system/instance templates, demos, history processors eliding old observations, 100k-char clipping, cache marks; no compaction	synchronous litellm.completion, tenacity 20 random-exponential retries, litellm fallbacks, no streaming	exactly one shell command per step in a SWE-ReX Docker bash session; blocklist; patch from /root/model.patch	step until submit or exit; cost, call and timeout budgets; requery on format errors; RetryAgent attempts with reviewer
OpenHands	Canvas posts user events to an Agent Server; SDK wraps them as MessageEvents; tool results are ObservationEvents	section-registry system prompt with AGENTS.md skills; event view condensed by LLM summary on token or size limit	litellm chat or Responses API; 5 exponential retries; profile fallbacks; optional token streaming	typed ToolDefinitions with executors in a local or Docker workspace; batch truncated at Finish; optional parallelism	status-machine loop with iteration and budget caps, stuck detector, risk-based confirmation, hooks, threaded delegation

	Input	Context	Model interface	Output	Control flow
Aider	terminal REPL, scripted message, or AI! file comments; results re-enter as plain text message pairs	ordered chunks: system, examples, PageRank repo map, files, weak-model summarized history; optional cache headers	LiteLLM completion with streaming, exponential backoff to 60 s, assistant-prefill continuation on length cutoff	text edit formats parsed and fuzzily applied to files, auto-commit, lint/test, approved unsandboxed shell commands	reflection loop up to 3 re-sends on edit/lint/test failure; confirm gates; architect-to-editor two-coder handoff
Cline	host runTurn with text, images, files; steer queue consumed per request; tool results as tool messages	templated system prompt plus rules, cloned transcript, hook context, estimator-driven compaction at 90% via summarizer	gateway over Vercel AI SDK streamText, native tools, 5 SDK retries plus 3 turn retries with backoff	native tool calls in parallel groups with timeouts; shell spawn, editor, MCP; git-ref checkpoints	loop until no tool calls or completion tool; maxIterations, mistake and loop trackers, approval gates, sub-agents
Gemini CLI	CLI/ACP parts with @file expansion; tool results and AskUser answers re-enter as functionResponse parts	snippet-composed system prompt, hierarchical GEMINI.md memory, curated history, 50% threshold snapshot compression, tool masking	@google/genai or Code Assist streaming, 10-attempt backoff, mid-stream retries, policy-chain model fallback	registry of core and MCP tools, parallel batches, node-pty shell, seatbelt/b-wrap/container sandboxes, 40k truncation	caller-driven tool loop, 100-turn cap, next-speaker and loop checks, policy/confirmation gate, hooks, classifier routing, sub-agents
Goose	user message persisted first; steer queue and elicitation answers; tool results batched into one message	templated system prompt plus hints, per-turn context event, 0.8 threshold summarization, tool-pair summaries, prompt caching	provider trait with required stream; reqwest SSE; 3 retries exponential; empty-response retries; no backup model	all tools are MCP extensions; concurrent select_all execution; optional Docker container; 50k/200k output spill	ordered operation pipeline with one inference step; 1000-turn cap; approval wait and rerun; stop hook; foreground sub-agents
OpenAI Agents SDK	string or Responses items plus Session history; tool outputs re-enter as run items; RunState resumes	instructions callable, tools and handoffs as functions, full item list; server compaction and trimmer	OpenAI Responses API via SDK; prefix-routed providers; configurable retry, timeout, streaming	function tools as concurrent asyncio tasks; hosted tools; Docker or local sandbox for shell and patches	NextStep loop with max_turns=10; guardrail tripwires; approval interruptions; handoffs swap agent in place
Google Agent Development Kit	types.Content user message appended as an Event; tool results and HITL answers re-enter as function-response events	ordered request processors rebuild LlmRequest from session events each step; instructions, memory tool, compaction, caching	BaseLlm adapters resolved by model name; Gemini via google-genai; SSE streaming; SDK retries; FallbackModel	BaseTool subclasses run concurrently as asyncio tasks with callbacks; code executors from in-process to Docker	Runner picks agent, flow loops until final response; max_llm_calls; confirmation pauses; transfers run sub-agents in place
AutoGen	chat messages from user or peers; tool results batched as one function-result message	one system message plus a pluggable ChatCompletionContext (unbounded, buffered, token-limited); memory hooks	ChatCompletionClient protocol; OpenAI SDK client with usage logging and streaming; SDK retries	workbench tools run concurrently via asyncio.gather; Docker or local code executors	bounded max_tool_iterations loop, then reflect or summarize; teams via actor runtime, termination conditions
CrewAI	Crew inputs interpolated into templates; tool results and human feedback appended as messages	role/goal/backstory system prompt, task plus memory and knowledge; LLM summarization near window limit	LLM class over native provider clients or litellm; rate-limit retries; streaming; native or ReAct text	in-process BaseTool execution with cache and repeat check; parallel native calls; no sandbox	Flow state machine per task to max_iter, task sequencing by crew, guardrail retries, human gate

	Input	Context	Model interface	Output	Control flow
smolagents	task string plus optional images and named args; tool results and errors re-enter as text observations	Jinja system prompt plus full step memory replayed each call; truncation only, no compaction	pluggable Model adapters over provider SDKs; rate-limit retries; native tool calls or code blobs	JSON tool calls run in threads, or Python snippets in an AST interpreter or remote sandbox	ReAct while-loop to max_steps, stops on final_answer; optional periodic planning; managed agents as tools
Cursor Agent	chat or CLI prompt, queued or steered messages delivered at next tool call, Ask-questions tool answers mid-work	rules at prompt start, skills on demand, local grep search, compaction near capacity, no native memory	Cursor backend to many providers, Auto routing by classifier, 256k to 1M windows, guardrail reroute	nine first-party tools plus MCP, Seatbelt or Landlock sandbox, network blocked by default, local checkpoints	unbounded tool loop, Run Mode gate with LLM classifier, two-level subagents, stop hook loops, coordinator Projects
GitHub Copilot cloud agent and agent mode	issues, @copilot mentions, chat handoff, API and automations; steering applied after the current tool call; write-access only	layered instruction files, custom agents, skills and validated 28-day memory; summarization not documented	GitHub-hosted Copilot API, Auto routing on health and task complexity, LTS fallback, AI-credit metering	ephemeral Actions runner with firewall, Bash plus read-only GitHub MCP, pushes to copilot/ branch, draft PR	background session with 59-minute cap, no in-session approvals, edge gates on triggers and workflows, automations

What was hard to place. The hard cases are collected in Appendix C, and four recur across the first set.

- **Primitives that call the model.** In Claude Code, opencode and pi, compaction decides what to summarize and then calls the model to summarize it; Claude Code's web-fetch tool and pi's code mode call models from inside a tool. Read as a call graph, these break the rule that Context and Output never call the model. Read as responsibility, they do not: the component still decides one thing, and uses Model interface to do it. We added the rule on primitives as services (Appendix B) for these cases.
- **Approval gates inside tools.** In opencode and Hermes the code that asks the person sits inside the tool that would act, and in Codex the approval requirement depends on what the sandbox allows. By function the gate is Control flow and the enforcement is Output; by location they are one file. The two also read each other, which is the interaction of concerns noted in §3.
- **Decisions carried out by editing the context.** opencode enforces a step budget by appending an assistant message and disabling tools; Hermes's stop gates append a synthetic message and go again; LangGraph replaces the model's answer with a canned message when the step budget is reached. Each is a Control flow decision whose mechanism is a change to what the model is shown. The decision and the mechanism have different homes, and we record both.
- **Nesting at the model interface.** Hermes can hand a whole turn to an external Codex process, or reach an agent over ACP as if it were a model. The thing behind Model interface is then another harness. The nested-harness rule covers it, but the nesting is at a different primitive from the sub-agent case, and the outer harness keeps only Input, persistence and a fallback.

In the wider set the same four recurred, and two more appeared. *Model calls whose only effect is a decision.* Gemini CLI's next-speaker and loop checks, Goose's permission judge, Cursor's auto-review classifier and AutoGen's reflection step each call the model and consume the answer as a Control flow decision. The nested-harness rule places them, and Cursor's classifier makes read-only tool calls of its own, so it is a small harness in fact. *The loop outside the client.* Gemini CLI's core makes one model call per invocation and its caller re-sends the tool results; OpenHands's loop, tools and client are spread over three repositories; Copilot's coding agent runs inside a GitHub

Actions job that is at once its executor and its time budget. The primitives are present in each case; they are not in one place. Two placements went by function against appearance: Aider's edit formats bundle the prompt that asks for a format (CONTEXT) with the parser and applier (OUTPUT) in one class, and smolagents' `final_answer` is a tool whose stop signal travels as an exception raised inside the sandbox, an OUTPUT mechanism carrying a CONTROL FLOW decision.

One finding is not a hard case but a boundary question. Claude Code's server-side classifier runs inside the API during the session's own requests, and opencode can delegate compaction to the provider. In the wider set the split is more common: the OpenAI Agents SDK can keep the conversation and compact it on the server, and hosted tools in the OpenAI Agents SDK, Google ADK and Cline (web search, a code interpreter) run inside the model call with no local OUTPUT step. In each, a function the vocabulary places in the harness runs on the provider's side of the API, and the local harness holds only a verdict, an opaque item or a result it did not produce. The primitives still apply: the harness is split across two machines, not reduced to four parts. But it means the boundary of §1 is at the model's weights, not at the API, and a deconstruction from a client's source alone will miss the part that runs behind the API. We found no component in the nineteen that satisfied none of the five definitions.

5.8 What the deconstruction does not show

Selection. Nineteen harnesses, most of them coding agents or frameworks used for coding examples. Harnesses for browsing, voice, robotics and customer service were not examined, and the design decisions in Table 2 are drawn from what these nineteen do. The selection criterion was wide use and the availability of sources; it is not a random sample.

Documentation-only analysis. Claude Code, Cursor and Copilot are deconstructed from documentation, which states what the harness does but not always how. Negative definitions are harder to confirm from documentation than from source, and a documented feature may differ from the shipped one.

The harness is not always in one place. Two of the six, and at least four of the thirteen, run part of their harness behind the provider's API. A deconstruction from client source or documentation describes the local part; the part on the provider's side is visible only through its effects. As providers absorb more harness functions, this gap will grow.

Products change. Every assignment is pinned to a commit or a documentation date. Several of these harnesses change weekly. The deconstruction describes them on those dates, and a reader checking it later should use the pinned versions.

Who made the assignments. The assignments were made by the author with an AI coding agent as a research instrument (see the disclosure), and each cites its evidence. No second, independent decomposition was made; §6 returns to this. The rules in Appendix B are where disagreement would most likely arise: they follow from the definitions, but another reader could draw some boundaries elsewhere.

Coverage is not comparison. That every component has a home shows the five cover what exists. It does not show that any implementation is better than another, which is the subject of §6.

6 Further Work: Comparing Implementations of a Primitive

The deconstruction shows that the five primitives cover what exists: every component in nineteen harnesses had a home in one primitive or one concern. It does not show which implementation of a primitive is better, for which model, on which task. That is the comparison the vocabulary makes well-posed, and it has not been done.

The form of the study. Hold the model fixed. Hold four primitives fixed. Vary the fifth, one design decision at a time, and measure on a task set with a check that does not depend on the primitive varied. The harness is stated in the five parts of Table 1, so that the varied part is the only difference between arms, and the statement is published with the result. Fan et al. [10] is the nearest existing study, with three decisions varied across four models; the vocabulary names the remaining decisions and says what must be held fixed.

One comparison per primitive. The decisions in Table 2 are the candidates. Within CONTEXT: window fitting, where the harnesses in §5 drop old turns, summarize, compact at a threshold with a verbatim tail, or leave it to the provider, and no two agree on when; and the placement of project instructions, preloaded or searched. Within OUTPUT: the action format, structured calls against model-written code run in an interpreter, which some of the nineteen offer and the rest do not; and the execution environment, host, OS sandbox or container, which changes what the model is allowed to do and what it is told it did. Within CONTROL FLOW: the stopping rule, a step budget against a cost budget against none; and the approval gate, a rule pipeline against a classifier model against per-tool assertion, measured on both task completion and the rate of harmful actions. Within MODEL INTERFACE: the retry and backup policy, measured under induced provider failure. Within INPUT: the delivery of mid-turn messages, at the next step boundary or after the turn, measured on tasks where the person changes the request part-way.

What such a study needs. A task set with a harness-independent check. A model held at a fixed version, since the harness that suits one model may not suit another [20]. The four fixed primitives reported in the same five-part schema, so that a later study can hold the same things fixed. And the evidence files of Appendix C are a starting point for the arms: each cell of Table 5 is an implementation that exists and can be ported into a common harness, because the primitives give the seam along which to port it.

These comparisons are also the test of the framework. The deconstruction tests coverage: can every component be placed? A comparison tests something the deconstruction cannot: can a result be attributed to the primitive varied? If an implementation cannot be placed in one primitive, or if varying one primitive changes behaviour only through another, the framework is wrong in a way that reading source could not show. The five questions of §4.3 make the same prediction in the small: a failure isolated to one primitive should be fixed by a change to that primitive alone. Both predictions are checkable, and neither has been checked at scale.

A second coding of the deconstruction. The assignments in Appendix C were made by the author with an AI coding agent as a research instrument, against the definitions of §2 and the rules of Appendix B. No independent second coding was made, so the agreement between two people applying the definitions is unmeasured. The brief, the definitions and the component lists are published for that purpose.

7 Conclusion

An agent is a model wrapped in a harness, and the harness has five parts: Input, Context, Model interface, Output and Control flow. Each is the smallest program that adds it, and each removes a failure the others do not. What distinguishes one harness from another is the design decisions made within those primitives, and what makes a harness fit to run is the answers it gives to six questions asked of each. Nineteen harnesses in wide use, six read from source at pinned versions and thirteen from their sources or documentation at summary level, fit this description with nothing left over. The vocabulary lets a result be reported with the harness that produced it, lets research compare primitives by their implementations, and lets an engineer isolate a failure to the primitives an implementation touches. The comparison of implementations is the work that remains, and it is also the test of the framework.

Disclosure of AI use

AI tools were used in two roles.

As a research instrument. The deconstruction of §5 was carried out with an AI coding agent (Claude, Anthropic) under the author's direction. The agent read each harness's source at the pinned commit, or its documentation on the stated date, applied the definitions of §2 and the rules of Appendix B, and recorded every assignment with a file-and-line or URL citation. The author reviewed the assignments and the hard cases. Every assignment in Appendix C can be checked against its cited source without the agent.

As a writing assistant. The author set the position, the structure, the definitions and the three-level vocabulary, and reviewed every section. An AI assistant drafted prose from the author's outline and the companion repository, and edited for consistency. The minimal harnesses of §2 and Appendix A, and the transcripts they cite, are the author's, recorded in the companion repository.

References

[1] All Hands AI. OpenHands and the OpenHands software agent SDK. https://github.com/All-Hands-AI/OpenHands, 2026. Commit aa593c7, 2026-10-16; SDK commit 5fa4dea, 2026-10-15.

[2] Anomaly and contributors. opencode. https://github.com/anomalyco/opencode, 2026. Commit 654b79b, 2026-10-15.

[3] Anthropic. Raising the bar on SWE-bench verified with Claude 3.5 Sonnet. https://www.anthropic.com/research/swe-bench-sonnet, 2025.

[4] Anthropic. Claude Code documentation. https://code.claude.com/docs/en/how-claude-code-works, 2026. Documentation as read on 2026-10-15.

[5] Anysphere. Cursor documentation. https://cursor.com/docs, 2026. Documentation as read on 2026-10-16.

[6] Block. goose. https://github.com/block/goose, 2026. Commit 5eca486, 2026-10-16.

[7] Cline Bot Inc. Cline. https://github.com/cline/cline, 2026. Commit 23791b9, 2026-10-15.

[8] CrewAI Inc. CrewAI. https://github.com/crewAIInc/crewAI, 2026. Commit 6115839, 2026-10-15.

[9] Chase Dovey. Building agents: From nothing to a model, a harness, and a model trained in its harness. https://github.com/averagejoeslab/building-agents, 2026. Book and companion repository, Average Joes Lab.

[10] Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents. *arXiv preprint arXiv:2609.20804*, 2026.

[11] Paul Gauthier. Aider: AI pair programming in your terminal. https://github.com/Aider-AI/aider, 2026. Commit 5dc9490, 2026-05-22.

[12] GitHub. GitHub Copilot coding agent documentation. https://docs.github.com/en/copilot, 2026. Documentation as read on 2026-10-16.

[13] Google. Gemini CLI. https://github.com/google-gemini/gemini-cli, 2026. Commit b25669d, 2026-10-16.

[14] Google. Agent development kit (Python). https://github.com/google/adk-python, 2026. Commit f78149a, 2026-10-15.

[15] Hugging Face. smolagents. https://github.com/huggingface/smolagents, 2026. Commit 96f33fa, 2026-10-06.

[16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

[17] Haibo Jin, Peng Kuang, Ye Yu, Xiaopeng Yuan, and Haohan Wang. Agent primitives: Reusable latent building blocks for multi-agent systems. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03695.

[18] Sayash Kapoor et al. Holistic agent leaderboard, 2025.

[19] LangChain. LangGraph. https://github.com/langchain-ai/langgraph, 2026. Commit 5eb60ae, 2026-10-14; documentation at https://docs.langchain.com/oss/python/langgraph/overview.

[20] METR. An update on our preliminary evaluations of Claude 3.5 Sonnet and o1. https://metr.org/blog/2025-01-31-update-sonnet-o1-evals/, 2025.

[21] METR. Measuring time horizon using Claude Code and Codex. https://evals.alignment.org/notes/2026-02-13-measuring-time-horizon-using-claude-code-and-codex/, 2026.

[22] Microsoft. AutoGen. https://github.com/microsoft/autogen, 2026. Commit 027ecf0, 2026-04-06.

[23] Model Context Protocol. Model Context Protocol: Architecture overview. https://modelcontextprotocol.io/docs/learn/architecture, 2026. Accessed 2026-10-15.

[24] Nous Research. Hermes Agent. https://github.com/NousResearch/hermes-agent, 2026. Commit 7bf3766, 2026-10-15.

[25] OpenAI. Codex CLI. https://github.com/openai/codex, 2026. Commit f0c5077, 2026-10-15.

[26] OpenAI. OpenAI agents SDK (Python). https://github.com/openai/openai-agents-python, 2026. Commit 17d994b, 2026-10-15.

[27] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

[28] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

[29] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2302.04761.

[30] Erik Schluntz and Barry Zhang. Building effective agents. https://www.anthropic.com/engineering/building-effective-agents, 2024.

[31] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2303.11366.

[32] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. arXiv:2309.02427.

[33] SWE-agent contributors. SWE-agent. https://github.com/SWE-agent/SWE-agent, 2026. Commit 3ea751c, 2026-07-16.

[34] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning*, 2024. arXiv:2402.01030.

[35] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. arXiv:2405.15793.

[36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. arXiv:2210.03629.

[37] J. Young. Effective harnesses for long-running agents. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, 2025.

[38] Mario Zechner. pi: a minimal coding agent. https://github.com/badlogic/pi-mono, 2026. Commit 5ab7fe7, 2026-10-15.

[39] Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026.

A The Minimal Harnesses

The five programs of §2, complete, without their dependency headers. Each is the previous one plus one primitive. They are `harness/1_model_interface.py` to `harness/quark.py` in the companion repository [9], and the transcripts quoted in §2 are from the recorded runs there. The companion repository adds the production concerns of §3 to the last program one question at a time, and later replaces `request_response` with a function that runs a local model and translates between the Messages format and the model's chat template, leaving the other four primitives unchanged.

A.1 Model interface alone

```
from anthropic import Anthropic

model = Anthropic()


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


response = request_response({"messages": [{"role": "user", "content": "The tests are failing. Find out why and
     fix it."}]})
print(response.model_dump_json(indent=2))
```

A.2 Plus Input

```
from anthropic import Anthropic

model = Anthropic()


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def capture_input():
    return input("\n> ")


response = request_response({"messages": [{"role": "user", "content": capture_input()}]})
print(response.model_dump_json(indent=2))
```

A.3 Plus Output

```
import subprocess
from anthropic import Anthropic

model = Anthropic()

bash = {"name": "bash", "description": "Run a shell command",
```

```
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["
     command"]}}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def capture_input():
    return input("\n> ")


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
            print(f"$ {block.input['command']}")
            ran = subprocess.run(block.input["command"], shell=True, capture_output=True, text=True)
            tool_results.append({"type": "tool_result", "tool_use_id": block.id, "content": ran.stdout + ran.
     stderr or "(no output)"})
    return tool_results


response = request_response({"tools": [bash], "messages": [{"role": "user", "content": capture_input()}]})
handle_output(response)
```

A.4 Plus Control flow

```
import subprocess
from anthropic import Anthropic

model = Anthropic()

bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["
     command"]}}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def capture_input():
    return input("\n> ")


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
            print(f"$ {block.input['command']}")
            ran = subprocess.run(block.input["command"], shell=True, capture_output=True, text=True)
            tool_results.append({"type": "tool_result", "tool_use_id": block.id, "content": ran.stdout + ran.
     stderr or "(no output)"})
    return tool_results


def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
```

```
            response = request_response({"tools": [bash], "messages": conversation})
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
                break                                   # done: hand back to the person
            conversation.append({"role": "user", "content": tool_results})   # a tool ran: go again


control_flow()
```

A.5 Plus Context: the complete harness

```
import subprocess, os, datetime
from anthropic import Anthropic

model = Anthropic()
bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["
     command"]}}


def capture_input():
    return input("\n> ")


def assemble_context(conversation):
    instructions = f"You are quark, an agent. You act through bash, in {os.getcwd()}. Today is {datetime.date.
     today()}."
    return {"system": instructions, "tools": [bash], "messages": conversation}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
            print(f"$ {block.input['command']}")
            ran = subprocess.run(block.input["command"], shell=True, capture_output=True, text=True)
            tool_results.append({"type": "tool_result", "tool_use_id": block.id, "content": ran.stdout + ran.
     stderr or "(no output)"})
    return tool_results


def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
            response = request_response(assemble_context(conversation))
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
                break                                   # done: hand back to the person
            conversation.append({"role": "user", "content": tool_results})   # a tool ran: go again


control_flow()
```

B Rules for the Cases at the Boundaries

The negative definitions of §2 decide most cases. The following rules decide the rest. Each was needed for a component found in §5.

Function, not location.
A component is assigned by what it does, not by the file or the name it has. Code inside a tool that decides what the model sees is Context. An approval decision made inside a tool's runtime is Control flow.

The tool channel.
The model drives other primitives through tool calls: spawning a sub-agent, entering a planning mode, writing a plan, searching for tools, asking the person a question. The call is Output; its effect is assigned to the primitive it changes.

Backup versus routing.
A model change triggered by a request that could not be served is Model interface. A model change keyed to the work is Control flow. A fixed model for one class of request, such as compaction summaries, is a request setting and Model interface.

Receiving versus rendering.
Receiving a response as a stream is Model interface; rendering each fragment is Output.

Parallel actions versus parallel calls.
Several tool calls from one response run concurrently: Output. Several model calls issued concurrently, to divide a task or to vote: Control flow.

The reader of a record.
A record kept for the model (a searchable transcript) is Context. A record kept for the operator (a trace) is observability. An artifact for the user (a diff view) is Output. One file may serve several readers, and each use is assigned separately.

The consumer of an answer.
An answer the harness consumes (an approval) is Control flow. An answer the model consumes (the result of an ask-the-user tool) is Input.

Direct actions by a person.
A command the person runs through the harness's own executor, whose output enters the conversation, is Input, although it reuses Output's executor.

Model-written programs.
A program the model writes at run time is Control flow if it sequences model calls and Output if it sequences only tools.

Bundles.
A mode, a skill, a hook system or an agent profile is packaging that bundles decisions across several primitives. Decomposed, each element has one home: a planning mode is a context template, a restricted tool set and an approval gate; a hook system is a dispatcher (Control flow) whose events each affect one primitive.

Nested harnesses.
A sub-agent is a harness of its own, and delegation to it is Control flow of the outer harness. The same holds for model-backed components inside a harness, such as an approval reviewer or a termination judge: each is Control flow with a Model interface call.

Primitives as services.
One primitive may use another's mechanism to do its job: Context asks the model to summarize old turns before compacting them; a web-fetch tool in Output asks the model to condense a page before returning it. The component is assigned by what it *decides* (what to compact, what to return), and the call it makes is a use of Model interface. The negative

definitions are about responsibility, not about which function calls which.

Tool protocols.
A protocol such as MCP [23] provides pluggable OUTPUT (third-party tools). Its other features are assigned by function: resources and prompts are CONTEXT; sampling borrows the client's MODEL INTERFACE; elicitation is INPUT.

C Evidence for the Deconstruction

Every assignment in §5, with its evidence, is published with the paper as a separate document, *Evidence for the Deconstruction* (`evidence.pdf`, about a hundred pages), generated from one Markdown file per harness in the repository under `paper/evidence/`. The first six harnesses are deconstructed in full, the thirteen of the wider set at summary level, two to five components per primitive. For harnesses with published source, a citation is a path and a line range at the pinned commit; for a harness read from documentation it is a page and section heading on the date read. Each harness's entry opens with its source, its pinned version and the path conventions used; *not found* means the mechanism was looked for and not found, not that it is absent. The brief the deconstruction followed, with the definitions, the rules and the output format, is published beside the evidence.

Table 2: Design decisions by primitive, and the choices seen across the harnesses deconstructed in §5.

Primitive	Decision	Choices seen
Input	source	terminal; editor; chat gateways; HTTP; issue tracker; scheduled trigger; a typed state channel
	mid-turn messages	delivered at the next step boundary (steer); after the turn (queue); as an interrupt; not at all
	the person's answers	consumed by the harness as approvals; returned to the model as tool results
Context	instructions	a system prompt alone; plus project files (`CLAUDE.md`, `AGENTS.md`); plus memory files; a sectioned prompt diffed into patches
	long-term memory	none; files preloaded into the instructions; a store searched by the agent through tools; summaries carried forward
	window fitting	drop old turns; summarize (prune, then summarize); compaction at a threshold with a verbatim tail; provider-native compaction; left to user code
	layout for caching	a stable prefix with changes appended; instruction changes frozen as dated events; byte-stable tiers
Model interface	client	provider SDK; in-house request compiled per protocol; one stream function over adapters; a library's chat-model call
	transport	HTTP with server-sent events; WebSocket, falling back to HTTP
	retries and backup	none; three to ten retries; retry split by whether output started; a fallback chain of models
	action format	native tool calls; calls parsed from text; a code block to execute
Output	tool set	one shell; a fixed built-in set; built-in plus MCP; plain tool objects; a tool node
	action format	structured calls; model-written code run in an interpreter (V8, QuickJS, in-process)
	execution environment	the host; an OS sandbox with a domain allowlist; a container; a remote sandbox
	parallelism	sequential; read-only calls concurrent; all calls concurrent as they stream
Control flow	loop shape	until no tool call; one task per turn; a user-written graph run as supersteps; a phase-verdict loop
	stopping rules	step budget; token or cost budget; unlimited; recursion limit; stop-gate nudges
	approval	a rule pipeline ending in a prompt; a classifier model; a reviewer model; each tool asserting its own permission
	delegation	an agent tool to a fixed depth; child sessions; sub-agents as full sessions; flat threaded delegation

Table 3: Six production concerns as questions asked of each primitive. Each cell names what the concern changes in that primitive; "—" means we found nothing essential there in the harnesses examined.

	Persistence *What outlasts the session?*	**Safety & security** *What could do harm, and who could make it?*	**Observability** *What did it do?*	**Resilience** *What if something breaks?*	**Performance** *What is slow or costly?*	**Evaluation** *Does it still work?*
Input	an unfinished session offered back	approvals asked of a person	refusals recorded	input ending; interrupts	—	yes to every approval in a test run
Context	memories; resumed transcripts	untrusted content; poisoned memory; roles and authority	—	compaction when too long	caching the unchanged prefix; trimming results	the instructions under test
Model interface	—	—	tokens, latency, cost per call	retries; a backup model	streaming	the model under test
Output	—	sandboxes; never-lists; secrets hidden; pinned images	what each action did	time limits on actions	parallel actions; smaller models for side tasks	the tools under test
Control flow	the task in progress; a diary of every message	runaway loops; the record kept out of reach	each decision	resume after a crash	—	the loop under test

Table 4: The five primitives, the step of the agent cycle each serves, and the CoALA component it corresponds to.

Primitive	Mechanism	In the agent cycle	CoALA
Input	capture requests and results	perceive	observations
Context	assemble what the model is shown	working and long-term memory	working memory; retrieval
Model interface	request a response	decide	reasoning (by the model)
Output	show words, carry out actions	act	grounding actions; learning (writes)
Control flow	go again, hand back, or stop	the cycle	decision cycle

Table 5: Six harnesses by primitive. Each cell is the design decision that harness made within the primitive; the evidence for each is in Appendix C.

	Claude Code	Codex CLI	opencode	pi	Hermes Agent	LangGraph
Input	prompts, stdin, queued mid-turn messages, scheduled prompts, sub-agent reports, tool results, answers to an ask-user tool	one operation channel per thread; steering queued mid-turn; mailbox; typed answers to the model's questions; tool results	durable inbox rows (person, shell, skill, synthetic), delivered as steer or queue at step boundaries	terminal, print and RPC modes into one prompt call; steer and follow-up queues	one message event from CLI, 25 messaging gateways, HTTP, ACP, cron; mid-turn steering	writes to a typed state channel, merged by a reducer: messages, tool results, resume values
Context	cached prefix (system, tools, CLAUDE.md) plus appended reminders; deferred MCP schemas; compaction re-injects memory and recent files	the Responses item list; world-state sections as diffs; project files walked to cwd; three compaction modes, with background compaction	event-sourced history re-projected per call; cached instructions as dated events; compaction at ∼90%, 15k tail, or provider-native	projection of a session tree; sectioned prompt diffed into patches; compaction past a reserve, 20k tail	three-tier byte-stable prompt; memory and project files; compaction at 50% by prune then summarize; tool-search bridge	the state's messages plus one system message; window-fitting left to a user hook
Model interface	Messages API, streaming; 10 retries; idle watchdogs; user-configured fallback chain	Responses API only; WebSocket, falling back to HTTP; 5 retries with server-advised timing; no server-side storage	in-house request compiled per protocol; SSE or WebSocket; retry split by whether output started	one stream function over ∼15 adapters; SDK retries off; two-tier retry; overflow routed to compaction	chat-completions client with pluggable transports; streaming; 3 retries, fallback chain, auto-recovery	one LangChain chat-model call per step; no client, streaming or fallback of its own
Output	built-in tools plus MCP; read-only calls concurrent; OS sandbox with domain allowlist; results spill to files; edit checkpoints	one orchestrator: approve, sandbox, run, retry on denial; OS sandboxes on three platforms; read–write gate for parallel tools; V8 code mode	tools run concurrently as they stream; each asserts its own permission; most reached through a code-mode interpreter	plain tool objects, parallel by default; no core sandbox; QuickJS code mode	core tools plus bridge; parallel-safe segments; sandboxed terminal (local, Docker, Modal, . . .); approval gate	tool node runs calls concurrently in-process; errors become messages; tools may return routing commands
Control flow	loop until no tool calls; six-step permission gate, with a classifier model by default; agent tool to depth 3; model-written workflows	task per turn; loop until no tool call; hooks, Guardian reviewer, then person; sub-agents as full sessions with depth caps	one fibre per session; continue while tools called; per-agent step budget; child sessions, depth 1	inner tool loop, outer follow-up loop, session loop for retries; virtual-model routing	phase-verdict loop; unlimited budget; stop-gate nudges; flat threaded delegation; task-keyed routing	a user-written graph run as supersteps; stop on empty task set or recursion limit; interrupts checkpointed